

An Analysis of RoBERTa

RoBERTa

RoBERTa -- Part 1

Token type: The token type is a standard embedding layer, translating a one-hot vector into a dense vector based on its token type. **Position:** The position embeddings are based on a token's position in the sequence. BERT uses absolute position embeddings, where each position in a sequence is mapped to a real-valued vector. Each dimension of the vector consists of a sinusoidal function that takes the position in the sequence as input. **Segment type:** Using a vocabulary of just 0 or 1, this embedding layer produces a dense vector based on whether the token belongs to the first or second text segment in that input. In other words, type-1 tokens are all tokens that appear after the [SEP] special token. All prior tokens are type-0. The three embedding vectors are added together representing the initial token representation as a function of these three pieces of information. After embedding, the vector representation is normalized using a LayerNorm operation, outputting a 768-dimensional vector for each input token. After this, the representation vectors are passed forward through 12 Transformer encoder blocks, and are decoded back to 30,000-dimensional vocabulary space using a basic affine transformation layer.

RoBERTa -- Part 2

History BERT was originally published by Google researchers Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. The design has its origins from pre-training contextual representations, including semi-supervised sequence learning, generative pre-training, ELMo, and ULMFit. Unlike previous models, BERT is a deeply bidirectional, unsupervised language representation, pre-trained using only a plain text corpus. Context-free models such as word2vec or GloVe generate a single word embedding representation for each word in the vocabulary, whereas BERT takes into account the context for each occurrence of a given word. For instance, whereas the vector for "running" will have the same word2vec vector representation for both of its occurrences in the sentences "He is running a company" and "He is running a marathon", BERT will provide a contextualized embedding that will be different according to the sentence. On October 25, 2019, Google announced that they had started applying BERT models to English-language search queries on Google Search within the US. On December 9, 2019, it was reported that BERT had been adopted by Google Search for over 70 languages. In October 2020, almost every single English-based query was processed by a BERT model.

RoBERTa -- Part 3

BERT is meant as a general pretrained model for various applications in natural language processing. That is, after pre-training, BERT can be fine-tuned with fewer resources on smaller datasets to optimize its performance on specific tasks such as natural language inference and text classification, and sequence-to-sequence-based language generation tasks such as question answering and conversational response generation. The original BERT paper published results demonstrating that a small amount of finetuning (for BERTLARGE, 1 hour on 1 Cloud TPU) allowed it to achieve state-of-the-art performance on a number of natural language understanding tasks:

RoBERTa -- Part 4

Given two sentences, the model predicts if they appear sequentially in the training corpus, outputting either [IsNext] or [NotNext]. During training, the algorithm sometimes samples two sentences from a single continuous span in the training corpus, while at other times, it samples two sentences from two discontinuous spans. The first sentence starts with a special token, [CLS] (for "classify"). The two sentences are separated by another special token, [SEP] (for "separate"). After processing the two sentences, the final vector for the [CLS] token is passed to a linear layer for binary classification into [IsNext] and [NotNext]. For example:

RoBERTa -- Part 5

In masked language modeling, 15% of tokens would be randomly selected for masked-prediction task, and the training objective was to predict the masked token given its context. In more detail, the selected token is:

RoBERTa -- Part 6

Architectural family The encoder stack of BERT has 2 free parameters: L , the number of layers, and H , the hidden size. There are always $H / 64$ self-attention heads, and the feed-forward/filter size is always $4H$. By varying these two numbers, one obtains an entire family of BERT models. For BERT:

RoBERTa -- Part 7

Masked language modeling (MLM): In this task, BERT ingests a sequence of words, where one word may be randomly changed ("masked"), and BERT tries to predict the original words that had been changed. For example, in the sentence "The cat sat on the [MASK]," BERT would need to predict "mat." This helps BERT learn bidirectional context, meaning it understands the

An Analysis of RoBERTa

RoBERTa

relationships between words not just from left to right or right to left but from both directions at the same time. Next sentence prediction (NSP): In this task, BERT is trained to predict whether one sentence logically follows another. For example, given two sentences, "The cat sat on the mat" and "It was a sunny day", BERT has to decide if the second sentence is a valid continuation of the first one. This helps BERT understand relationships between sentences, which is important for tasks like question answering or document classification.

RoBERTa -- Part 8

replaced with a [MASK] token with probability 80%, replaced with a random word token with probability 10%, not replaced with probability 10%. The reason not all selected tokens are masked is to avoid the dataset shift problem. The dataset shift problem arises when the distribution of inputs seen during training differs significantly from the distribution encountered during inference. A trained BERT model might be applied to word representation (like Word2Vec), where it would be run over sentences not containing any [MASK] tokens. It is later found that more diverse training objectives are generally better. As an illustrative example, consider the sentence "my dog is cute". It would first be divided into tokens like "my1 dog2 is3 cute4". Then a random token in the sentence would be picked. Let it be the 4th one "cute4". Next, there would be three possibilities:

RoBERTa -- Part 9

Tokenizer: This module converts a piece of English text into a sequence of integers ("tokens"). Embedding: This module converts the sequence of tokens into an array of real-valued vectors representing the tokens. It represents the conversion of discrete token types into a lower-dimensional Euclidean space. Encoder: a stack of Transformer blocks with self-attention, but without causal masking. Task head: This module converts the final representation vectors into one-shot encoded tokens again by producing a predicted probability distribution over the token types. It can be viewed as a simple decoder, decoding the latent representation into token types, or as an "un-embedding layer". The task head is necessary for pre-training, but it is often unnecessary for so-called "downstream tasks," such as question answering or sentiment classification. Instead, one removes the task head and replaces it with a newly initialized module suited for the task, and finetune the new module. The latent vector representation of the model is directly fed into this new module, allowing for sample-efficient transfer learning.